

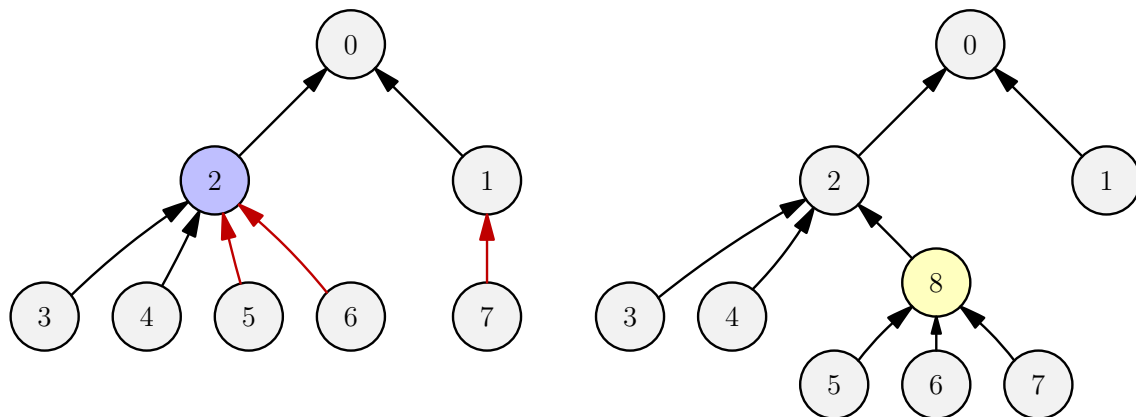
Masters and Slaves (proxies)

William has finally solved the nasty “L/10-L/16-bipasso” problem, and is now ready to *start earning money immediately!* For this noble purpose, he arranged his N fully functional computers (numbered from 0 to $N - 1$) into a tree-shaped network rooted at server 0, so that computer i is a “slave” of computer M_i (conventionally, we assume that $M_0 = -1$). Now the most demanding tasks from all over the world are being delivered to his farm, and will eventually be completed (hopefully)!



Figure 1: William connecting a newly-recycled proxy.

Sadly, some of his computers are already experiencing severe heating problems, not being able to handle the output of all of their slaves. Since the recycling bin for electronic waste is always filling up, William can promptly solve this emergencies through *proxying*: when some computers are overheating, William selects K of their slaves $S_0, \dots, S_{K-1}$ and *delegates* those to a newly-recycled proxy. Afterwards, the proxy is connected to another computer C .



In this case, computers 1 and 2 are overheating (left) thus William delegates $K = 3$ of their slaves (computers 5, 6, and 7) to the new proxy 8, connecting it to computer $C = 2$. Even though this reconfiguration can solve the overheating problem, William is worried that the tree structure would grow too *high*,¹ slowing down the computations. What is worse, a wrong reconfiguration could even create a *cycle*² causing an infinite data flow!

Given a sequence of E reconfigurations, help William keep track of the situation by determining whether each emergency causes the height to increase or, even worse, creates a cycle.

📎 Among the attachments of this task you may find a template file `proxies.*` with a sample incomplete implementation.

¹The *height* of a rooted tree is measured as the maximum number of links that are necessary to reach the root from a node: the sample graphs have height 2 (left) and 3 (right).

²A *cycle* is a circular sequence of computer, where each of them is a slave of the next.

Input

The first line contains integers N, E . The second line contains integers $M_0, M_1, \dots, M_{N-1}$. Each of the following E lines contains integers C, K followed on the same line by integers $S_0, S_1, \dots, S_{K-1}$.

Output

You need to write a single line with a sequence of characters, such that the i -th character is:

- ‘c’: if the i -th reconfiguration creates a cycle;
- ‘1’: if the i -th reconfiguration causes the tree height to increase;
- ‘0’: if the i -th reconfiguration does not cause the tree height to increase.

You need to output a character **until** either a cycle is created or the reconfigurations are finished. In other words, the sequence can either consist of E characters among ‘0’ and ‘1’ or less than E characters among ‘0’ and ‘1’ followed by a single ‘c’.

Constraints

- $1 \leq N \leq 1\,000\,000$.
- $1 \leq E \leq 10\,000$.
- $0 \leq C, S_i, M_i < N$ for each $i = 1 \dots N - 1$, $M_0 = -1$.
- $0 \leq K \leq 100$ in each emergency.
- The master/slave structure forms a tree (there are no circular links).
- The height of the tree never grows above 100.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

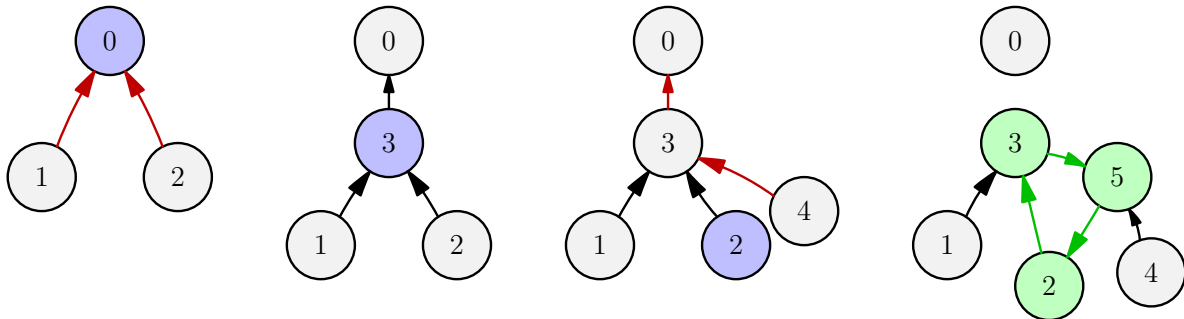
- **Subtask 1** [5 points]: Examples.
- **Subtask 2** [25 points]: $N, E \leq 10$.
- **Subtask 3** [30 points]: $N \leq 1000$.
- **Subtask 4** [20 points]: S_i ’s master is C , for each i and reconfiguration.
- **Subtask 5** [20 points]: No additional limitations.

Examples

input.txt	output.txt
<pre> 3 5 -1 0 0 0 2 1 2 3 0 2 2 4 3 1 1 0 6 5 4 2 3 1 5 </pre>	<pre> 10 c </pre>
<pre> 8 4 -1 0 0 2 2 2 2 1 2 3 5 6 7 2 2 5 4 8 2 6 7 1 0 </pre>	<pre> 10 10 </pre>

Explanation

In the **first sample case**, the network structure evolves as follows:



After the third reconfiguration, a cycle is detected hence the following reconfigurations do not take place.

The **second sample case** corresponds to the structure described in the text, which evolves as follows:

